

MEDIBOOK - RESUME TOTAL DU PROJET

Guide de comprehension, de reproduction et de progression personnelle

Date: 25 mai 2026

1. OBJECTIF GLOBAL DU PROJET

MediBook est une application web Django de gestion medicale. Le projet centralise:

- les medecins et leurs specialites,
- les patients,
- les disponibilites des medecins,
- la reservation, la modification et l'annulation des rendez-vous,
- l'historique des notifications liees aux rendez-vous,
- un tableau de bord global,
- une orientation "IA" simple pour suggerer une specialite a partir de symptomes.

L'idee generale du projet est de simuler une plateforme de coordination medicale complete, avec une logique metier reelle et une structure assez proche d'un petit produit deployable.

2. STACK TECHNIQUE UTILISEE

Back-end:

- Python
- Django 6.0.5

Base de donnees:

- SQLite par default en local
- PostgreSQL prevu pour Docker
- Support MySQL egalement dans les settings

Front-end:

- Templates Django
- Bootstrap
- CSS personnalise
- jQuery / Popper / Bootstrap JS

Media et fichiers:

- Pillow pour la gestion des images

Execution / deploiement:

- Unicorn
- Docker
- Docker Compose

CI:

- GitHub Actions

3. ARCHITECTURE GENERALE DU PROJET

Le projet est organise autour de plusieurs apps Django:

- accounts:
app presente mais encore minimale. Elle contient surtout une vue de test.
- patients:
gere les profils patients et l'affichage de leur historique de rendez-vous.
- doctors:
gere les specialites et les medecins.

- schedules:
gere les disponibilites des medecins, donc les creneaux reservables.
- appointments:
gere la logique principale des rendez-vous.
- notifications:
conserve les notifications creees apres des actions importantes.
- dashboard:
calcule les statistiques globales pour la page d'accueil.
- ai_orientation:
propose une orientation vers une specialite en fonction des symptomes saisis.

4. CONFIGURATION DU PROJET DJANGO

Le fichier principal de configuration est:

- medibook/medibook/settings.py

Ce qu'il faut retenir:

- les variables d'environnement sont utilisees pour la cle secrete, le mode debug, les hosts autorises et la base de donnees;
- le projet peut fonctionner avec SQLite, PostgreSQL ou MySQL;
- la langue est configuree en francais;
- le fuseau horaire par default est Africa/Casablanca;
- les dossiers static et media sont prepares.

Pourquoi c'est important:

- tu apprends a rendre un projet configurable selon l'environnement;
- tu separes le code de la configuration;
- tu prevois deja un passage du local vers la production.

5. ROUTAGE PRINCIPAL

Le fichier medibook/medibook/urls.py connecte les apps suivantes:

- /accounts/
- /patients/
- /doctors/
- /appointments/
- /schedules/
- /dashboard/
- /ai_orientation/
- /notifications/

La racine "/" pointe vers le dashboard global.

Le projet sert aussi les fichiers statiques et media via des routes dediees.

6. MODELES DE DONNEES ET RELATIONS

6.1 Patients

Modele:

- ProfilPatient

Champs:

- nom
- prenom
- email unique
- telephone
- date_de_naissance

Compétence acquise:

- création d'un modèle métier simple et propre;
- utilisation d'un `__str__` utile;
- gestion d'un email unique.

6.2 Medecins et specialites

Modèles:

- Specialite
- Medecin

Specialite contient:

- name
- description

Medecin contient:

- nom
- prenom
- email unique
- telephone_professionnel
- specialite (ForeignKey)
- adresse_cabinet
- statut_actif
- created_at
- photo

Compétences acquises:

- relation un-à-plusieurs via ForeignKey;
- usage de `related_name` pour retrouver les medecins d'une specialite;
- ajout d'une image avec `ImageField`;
- filtrage des medecins actifs seulement.

6.3 Disponibilites

Modèle:

- Disponibilite

Champs:

- medecin
- date
- heure_debut
- heure_fin
- est_reserve

Règle importante:

- contrainte d'unicité sur (medecin, date, heure_debut)

Pourquoi c'est fort:

- tu ne pars pas directement d'un rendez-vous libre;
- tu modélises d'abord les créneaux disponibles, ce qui est une vraie bonne logique métier.

6.4 Rendez-vous

Modèle:

- RendezVous

Champs:

- patient
- medecin
- specialite

- date
- heure
- motif
- statut
- date_creation

Statuts:

- en_attente
- confirme
- annule
- termine
- absent

Contrainte importante:

- un medecin ne peut pas avoir deux rendez-vous au meme creneau.

Validation metier dans clean():

- la specialite choisie doit correspondre a celle du medecin;
- un conflit est refuse si le medecin a deja un autre rendez-vous non annule au meme moment;
- le rendez-vous doit correspondre a une disponibilite existante.

Point tres important pour ta progression:

ce projet montre bien la difference entre:

- validation front via formulaire,
- validation back via modele,
- protection supplementaire via contrainte SQL/Django.

6.5 Notifications

Modele:

- Notification

Champs:

- rendez_vous
- type_notification (email ou sms)
- message
- envoye_le
- statut_succes

Ici la notification est surtout historique. Elle n'envoie pas encore un vrai email ni un vrai SMS, mais elle enregistre l'evenement comme si le systeme traçait les communications.

6.6 Orientation IA

Modele:

- AnalyseSymptome

Champs:

- patient
- texte_symptomes
- specialite_recommandee
- score_confiance
- cree_le

Ce module ne fait pas appel a un vrai modele de machine learning avance. Il s'appuie sur un dictionnaire de mots-cles par specialite.

C'est simple, mais pedagogiquement utile:

- tu apprends a integrer une logique "intelligente" dans un workflow web;
- tu stockes le resultat en base;

- tu affiches un historique.

7. LOGIQUE METIER PAR APP

7.1 doctors

Fonctionnalites:

- lister les medecins actifs;
- afficher le detail d'un medecin;
- lister les specialites;
- afficher une specialite avec ses medecins actifs.

Ce que tu apprends:

- get_object_or_404;
- select_related sur les relations;
- vues simples mais propres;
- filtration par statut.

7.2 patients

Fonctionnalites:

- lister les patients;
- afficher le detail d'un patient;
- afficher son historique de rendez-vous.

Ce que tu apprends:

- navigation a partir d'un patient vers ses rendez-vous;
- tri chronologique;
- lecture de l'historique d'un parcours de soin.

7.3 schedules

Fonctionnalite:

- afficher le planning libre d'un medecin.

Ce que tu apprends:

- separation entre disponibilites et rendez-vous effectifs;
- affichage de creneaux libres uniquement.

7.4 appointments

C'est le coeur du projet.

Fonctionnalites:

- reserver un rendez-vous;
- lister les rendez-vous du patient courant;
- modifier un rendez-vous;
- annuler un rendez-vous.

Logique cle:

- un patient courant de demonstration est cree automatiquement s'il n'existe pas ;
- lors d'une reservation, le rendez-vous est enregistre puis le creneau correspondant passe a reserve;
- une notification est creee;
- lors d'une modification, l'ancien creneau est libere puis le nouveau est reserve;
- lors d'une annulation, le statut passe a "annule" et le creneau redevient libre.

Competences tres utiles:

- ModelForm;

- clean() personnalise sur formulaire;
- messages Django;
- synchronisation entre deux modeles relies;
- protection contre les reservations incoherentes.

7.5 notifications

Fonctionnalite:

- lister les notifications trieées de la plus recente a la plus ancienne.

Ce que tu apprends:

- select_related sur plusieurs niveaux;
- creation d'un historique lisible pour les actions systeme.

7.6 dashboard

Fonctionnalite:

- afficher des statistiques globales:
 - total medecins,
 - total patients,
 - total rendez-vous,
 - rendez-vous en attente,
 - rendez-vous confirmes,
 - rendez-vous annules.

Ce que tu apprends:

- agregations simples;
- restitution metier sous forme de KPI.

7.7 ai_orientation

Fonctionnalites:

- analyser un texte de symptomes;
- recommander une specialite;
- calculer un score de confiance;
- conserver l'historique des analyses.

Logique:

- chaque specialite cle est liee a une liste de mots-cles;
- on compte les mots correspondants dans le texte;
- on retient le meilleur score;
- si aucun score n'est fort, une specialite par default peut etre choisie.

Ce que tu apprends:

- traitement de texte simple;
- logique de scoring;
- persistance d'une recommandation;
- interface entre "mini IA" et application metier.

8. INTERFACES ET EXPERIENCE UTILISATEUR

Le projet ne s'est pas limite au back-end.

Points visibles:

- un template de base commun;
- une barre de navigation reliee a toutes les fonctionnalites;
- un dashboard d'accueil plus travaille visuellement;
- un theme personnalise avec CSS sur mesure;
- personnalisation de l'admin Django.

Le CSS montre un effort de design:

- palette medicale douce;

- effet de profondeur;
- composants de type cartes, badges et statistiques;
- typographie plus soignée que du Bootstrap brut.

Compétence acquise:

- savoir prendre un projet Django fonctionnel et lui donner une vraie identité visuelle.

9. ADMIN DJANGO

Plusieurs modèles sont enregistrés dans l'admin:

- AnalyseSymptome
- RendezVous
- Specialite
- Medecin
- Notification
- ProfilPatient
- Disponibilite

Des `list_display`, filtres et recherches sont configurés.

Pourquoi c'est important:

- l'admin devient un vrai outil de pilotage;
- tu apprends à exploiter Django Admin au lieu de le laisser brut.

10. TESTS ET QUALITE

Tests utiles déjà présents:

- validation du formulaire de rendez-vous selon la disponibilité;
- création d'une analyse d'orientation IA via la vue.

Constat honnête:

- plusieurs apps ont encore des fichiers `tests.py` vides;
- la couverture de tests n'est donc pas complète.

Ce que cela veut dire pour ton niveau:

- tu as déjà commencé les bons réflexes;
- l'étape suivante est d'écrire des tests pour les vues, les modèles, les contraintes et les cas limites.

11. DOCKER ET EXECUTION

Dockerfile:

- base python:3.13-slim
- installation des dépendances
- copie du projet
- lancement via Gunicorn

`docker-compose.yml`:

- service web
- service db en PostgreSQL 16
- migration automatique au démarrage
- exposition du port 8000

Compétences acquises:

- containeriser une application Django;
- séparer application et base de données;
- préparer un lancement plus proche de la production.

12. CI / AUTOMATISATION

Le workflow `.github/workflows/ci-cd.yml` fait:

- checkout du code;
- installation de Python 3.13;
- installation des dependances;
- verification qu'aucune migration oubliee n'est en attente;
- execution de "python manage.py check".

Ce que tu apprends:

- automatiser des controles de base a chaque push ou pull request;
- integrer la qualite dans le cycle de developpement.

Note:

le workflow s'appelle CI et pas encore vraiment CD, car il ne deploie pas l'application. Il valide surtout la configuration et l'etat des migrations.

13. FORCES DU PROJET

- architecture modulaire par apps Django;
- logique metier realiste autour des disponibilites et rendez-vous;
- validations coherentes a plusieurs niveaux;
- presence de Docker et d'une CI;
- personnalisation visuelle;
- debut d'automatisation des tests;
- integration d'un module d'orientation de type IA.

14. LIMITES ACTUELLES A CONNAITRE

- l'app accounts n'est pas encore developpee comme un vrai systeme d'authentification;
- le patient courant est un patient de demonstration cree automatiquement;
- les notifications ne sont pas reliees a un service email/SMS reel;
- l'orientation IA repose sur des mots-cles simples, pas sur un modele medical avance;
- les tests restent incomplets;
- les permissions et roles utilisateurs ne sont pas encore au centre du projet.

Connaitre les limites fait partie de la maitrise:

un bon developpeur sait distinguer ce qui est deja robuste de ce qui reste a industrialiser.

15. CE QUE TU AS VRAIMENT APPRIS EN CONSTRUISANT CE PROJET

Tu as travaille, de facon concrete, les competences suivantes:

- modelisation de donnees relationnelles avec Django ORM;
- creation d'apps separees avec responsabilites claires;
- gestion des relations entre modeles;
- creation de vues et de templates;
- utilisation des ModelForm;
- mise en place de contraintes d'unicite;
- validations metier personnalisees;
- synchronisation entre disponibilites et rendez-vous;
- utilisation des messages utilisateur;
- personnalisation de l'admin Django;
- organisation des fichiers statiques;
- containerisation avec Docker;
- verification continue avec GitHub Actions;
- ecriture de premiers tests automatises;
- integration d'une logique d'orientation semi-intelligente.

16. COMMENT REFAIRE CE PROJET SEUL

Voici une methode tres pratique pour le refaire sans aide:

Etape 1:
créer un projet Django propre, puis des apps séparées:
patients, doctors, schedules, appointments, notifications, dashboard, ai_orientation.

Etape 2:
modéliser d'abord les entités centrales:

- Spécialité
- Médecin
- ProfilPatient
- Disponibilité
- RendezVous

Etape 3:
ajouter les relations et contraintes:

- un médecin appartient à une spécialité;
- un rendez-vous lie patient, médecin et spécialité;
- une disponibilité doit être unique par médecin/date/heure_debut;
- un rendez-vous doit être unique par médecin/date/heure.

Etape 4:
développer les vues de lecture:

- liste médecins;
- détail médecin;
- liste spécialités;
- liste patients;
- détail patient;
- planning médecin.

Etape 5:
développer la logique CRUD des rendez-vous:

- réservation;
- liste;
- modification;
- annulation.

Etape 6:
lier les rendez-vous aux disponibilités:

- réserver un slot quand un RDV est créé;
- libérer l'ancien slot en cas de modification;
- libérer le slot en cas d'annulation.

Etape 7:
ajouter les notifications et l'historique.

Etape 8:
créer le dashboard avec des compteurs globaux.

Etape 9:
ajouter l'orientation IA simple à base de mots-clés.

Etape 10:
finaliser avec:

- templates communs,
- CSS,
- admin,
- tests,
- Docker,
- CI.

17. PLAN DE MAITRISE PERSONNELLE

Si tu veux vraiment maitriser ce projet, travaille dans cet ordre:

1. Maitriser les modeles et les relations ORM.
2. Refaire les vues CRUD sans regarder le code.
3. Refaire les formulaires avec validation personnalisee.
4. Refaire les templates de base et la navigation.
5. Refaire Docker et docker-compose de memoire.
6. Refaire le workflow GitHub Actions.
7. Ecrire toi-meme de nouveaux tests.
8. Remplacer le patient de demo par une vraie authentication.
9. Brancher de vraies notifications email.
10. Ameliorer l'orientation IA avec une logique plus riche.

18. AMELIORATIONS CONSEILLEES POUR PASSER AU NIVEAU SUPERIEUR

- ajouter un vrai systeme d'inscription / connexion;
- distinguer les roles patient, medecin, admin;
- empecher l'acces d'un patient aux rendez-vous d'un autre;
- envoyer de vrais emails;
- permettre la confirmation reelle des rendez-vous;
- ajouter pagination et recherche;
- renforcer les tests unitaires et d'integration;
- ajouter des API REST avec Django REST Framework;
- securiser davantage la configuration de production;
- connecter un vrai modele NLP pour l'orientation IA.

19. CONCLUSION

Ce projet est deja une base serieuse pour montrer que tu sais construire une application web metier complete avec Django.

Tu n'as pas seulement fait des pages:

tu as mis en place une logique de donnees, des regles metier, des formulaires, des validations, un tableau de bord, un design coherent, une conteneurisation et une verification continue.

Autrement dit, tu as deja touche a plusieurs competences de niveau professionnel :

- architecture,
- back-end,
- base de donnees,
- UX simple,
- automatisation,
- qualite logicielle.

Le meilleur moyen de vraiment maitriser tout cela maintenant est de refaire le projet par blocs, sans copier, en t'obligeant a expliquer chaque choix:

"Pourquoi ce modele ?"

"Pourquoi cette relation ?"

"Pourquoi cette validation ?"

"Pourquoi cette contrainte ?"

"Pourquoi Docker ici ?"

"Pourquoi la CI ici ?"

Si tu arrives a reconstruire MediBook seul en expliquant chaque decision, alors tu ne seras plus en train de suivre un projet: tu seras en train de maitriser ton metier.